Object Model and Runtime Architecture of the NMR Shielding Extractor

Generated/Reviewed Documentation (Codex/Opus/Human) for v0.0.1

Contents

1 Authority and scope

This document describes the object model and runtime architecture used by the NMR chemical-shielding extractor. The design documents `OBJECT_MODEL.md`, `PATTERNS.md`, and `spec/CONSTITUTION.md` state the intent: typed objects, no runtime string dispatch for chemical identity, geometry separated from invariant protein identity, and full rank-2 tensor output. The source is the authority for the exact type names, ownership, and call order.

The code read for this document is: `Protein`, `ProteinConformation`, `ProteinTopology`, `ConformationResult`, `DenseBuffer`, `OperationRunner`, `OperationLog`, `Session`, `Trajectory`, `TrajectoryProtein`, and `RunConfiguration`, with the adjacent headers that define the actual per-atom records: `ConformationAtom`, `TrajectoryAtom`, `TrajectoryResult`, `Atom`, `Residue`, `Bond`, `Ring`, and `LegacyAmberTopology`.

The reader should keep one boundary in mind:

$$\text{Protein identity/topology} \quad \neq \quad \text{conformation geometry} \quad \neq \quad \text{trajectory history.}$$

The code enforces this with separate objects and separate ownership.

2 The system in one page

The architecture is shaped around one separation: a protein's identity and topology — its atoms, residues, bonds, and rings — are fixed once built, while everything that depends on coordinates is not. Identity is held in one invariant object; a conformation holds a coordinate set and the results computed from it; a trajectory holds the history across frames. Ownership, call order, and buffer placement follow from keeping these three apart.

The extractor is a one-way accumulation system. A loader builds a `Protein`; a conformation supplies one coordinate set for that protein; `OperationRunner::Run` attaches named `ConformationResult` singletons to that conformation in dependency order; trajectory mode repeats that conformation-level run for streamed frames and lets `TrajectoryResult` objects accumulate over the frame sequence.

2.1 Primary objects

`Protein` The invariant protein. It owns atoms, residues, the topology object, force-field charge table, build context, and any permanent conformations. It holds no atom coordinates and no computed shielding fields.

`LegacyAmberTopology` The concrete topology attached to `Protein`. It owns `CovalentTopology`, `RingTopology`, force-field invariant fields captured at the loader boundary, and the per-atom typed chemistry substrate `AtomSemanticTable`.

`ProteinConformation` One coordinate set for a protein. It owns the position vector, the parallel `ConformationAtom` vector, derived geometry caches such as ring geometries and bond midpoints, and the attached `ConformationResult` map.

`ConformationAtom` The per-atom computed-data record for one conformation. It stores position and the typed fields written by calculators: charges, neighbour records, tensors, spherical decompositions, solvent fields, DFT tensors, and feature-side values. It does not duplicate element or residue identity and does not own topology; bond and ring references appear only inside computed neighbourhood records.

`ConformationResult` The named operation object for conformation-scope computation. It declares a name, type-index dependencies, and optional feature writing. Subclasses own the physics query surface and write their results into `ConformationAtom` fields or into result-owned state.

`TrajectoryProtein` The trajectory-scope protein. It wraps the owned `Protein`, seats frame 0 as a permanent `MDFrameConformation`, allocates one `TrajectoryAtom` per protein atom, owns attached `TrajectoryResult`s, and receives dense buffers at finalization.

`TrajectoryAtom` The per-atom trajectory record. It holds named Welford state substructs and an atom-scope `RecordBag<AtomEvent>`. It does not hold position or identity.

`Trajectory` The run process and post-run record. It owns source paths, frame times, frame indices, the single-slot `TrajectoryEnv`, the run-scope `RecordBag<SelectionRecord>`, and the `GromacsFrameHandler` while the run is active.

`RunConfiguration` A typed run shape: per-frame `RunOptions`, deferred `TrajectoryResult` factories, required per-frame `ConformationResult` types, AIMNet2 requirement, and stride.

2.2 Static and trajectory flows

The static conformation flow is short:

```
loader builds Protein
Protein::FinalizeConstruction(positions)
Protein::AddConformation / AddCrystalConformation / AddPrediction / AddMDFrame
OperationRunner::Run(conf, RunOptions)
ConformationResult::WriteAllFeatures(conf, output_dir)
```

The trajectory flow keeps the same per-frame computation and adds one outer streaming layer:

```
TrajectoryProtein::BuildFromTrajectory(dir)
Trajectory::Run(tp, config, session, extras, output_dir)
  open GromacsFrameHandler
  read frame 0
  tp.Seed(first_frame_positions, time_ps)
  attach TrajectoryResults from RunConfiguration
  validate dependencies and Session resources
  build per-frame RunOptions
  emit invariant atom/topology sidecars if output_dir is set
  frame 0:
    update TrajectoryEnv and frame RunOptions
    conf0 = tp.MutableCanonicalConformation_()
    OperationRunner::Run(conf0, opts)
    tp.DispatchCompute(conf0, traj, 0, time)
  each later dispatched frame:
    skip/read according to stride
    update TrajectoryEnv and frame RunOptions
    conf = tp.TickConformation(positions)
    OperationRunner::Run(*conf, opts)
    tp.DispatchCompute(*conf, traj, frame_index, time)
  tp.FinalizeAllResults(traj)
```

Frame 0 is permanent and is an `MDFrameConformation`. Later streamed frames are free-standing base `ProteinConformation` instances returned by `TrajectoryProtein::TickConformation`; they point at the wrapped `Protein` for topology and die at the end of the loop iteration. There is no separate per-frame geometry class.

3 Diagrams

The diagrams in this section were rendered from Mermaid sources with `mmdc` and the Puppeteer configuration in `doc/calculators/.puppeteerrc.json`. They are included as PDFs so the compiled document is self-contained.

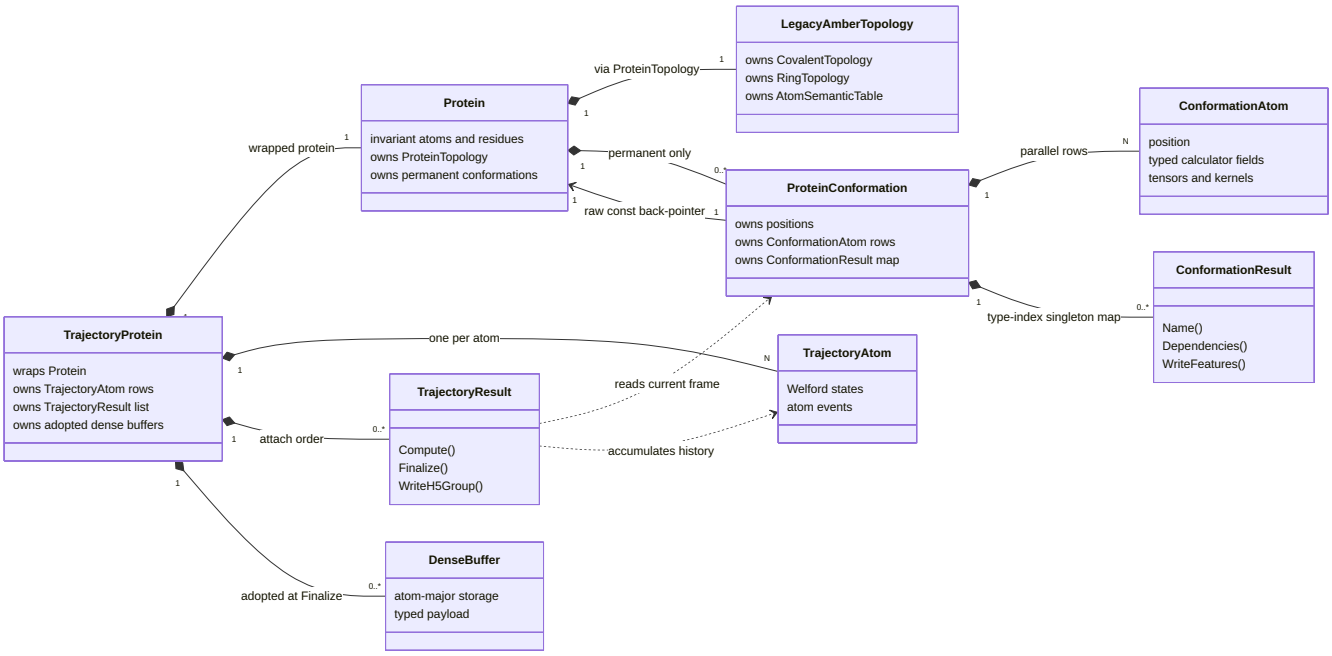

Figure 1: Object-model ownership and dependency relationships. Permanent conformations are owned by `Protein`; later streamed frames are temporary `ProteinConformation` objects owned by the trajectory loop.

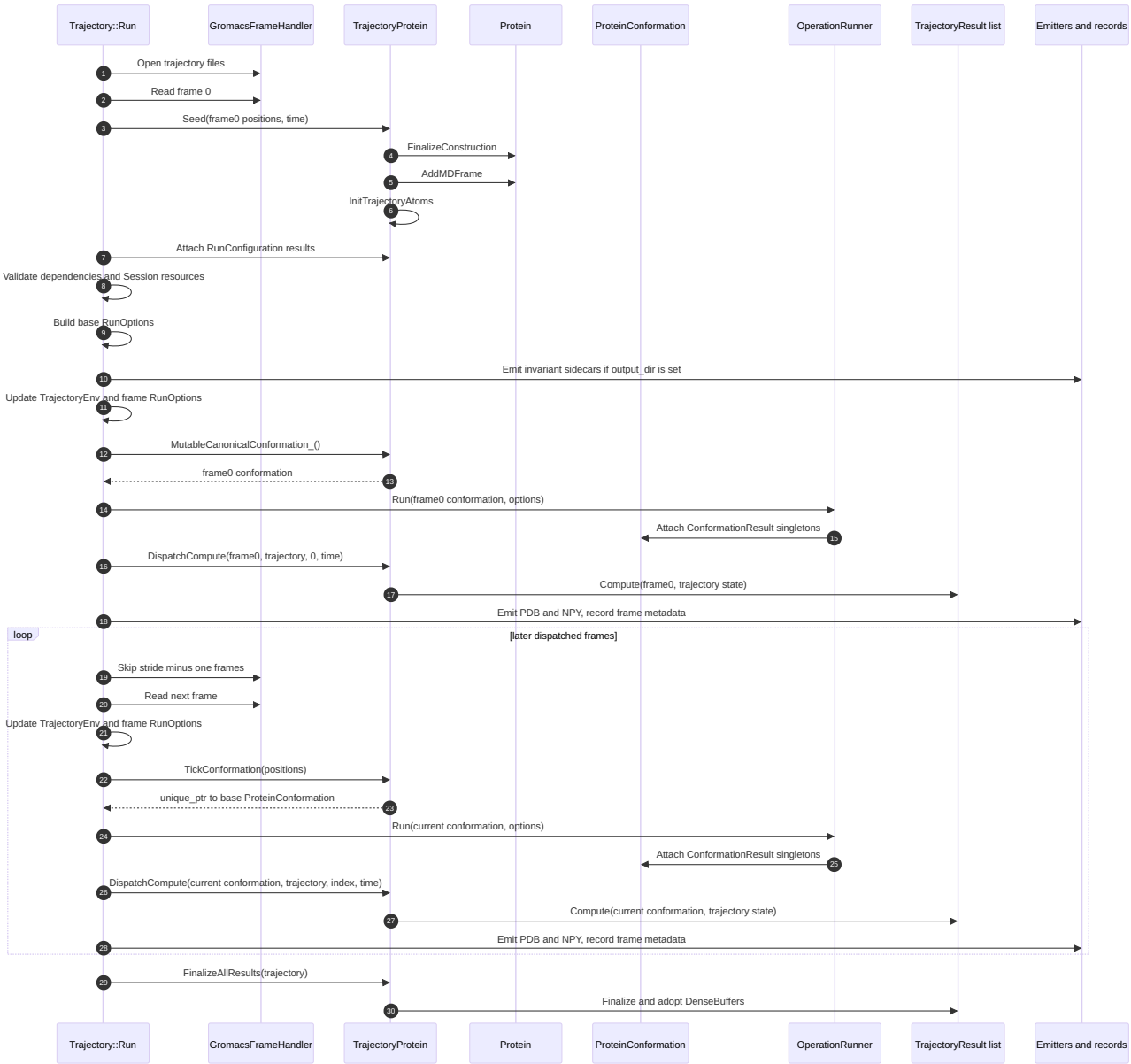

Figure 2: Per-frame trajectory call flow, showing the permanent frame-zero conformation and the temporary later-frame conformations.

4 Protein identity and topology

`Protein` is non-copyable and non-movable. The reason is concrete: `ProteinConformation` stores a raw `const Protein*` back-pointer, and deleting move construction prevents that pointer from being invalidated by a moved parent.

`Protein` owns these member groups:

```
std::vector<std::unique_ptr<Atom>> atoms_;
std::vector<Residue> residues_;
std::unique_ptr<ProteinTopology> protein_topology_;
std::unique_ptr<ForceFieldChargeTable> force_field_charges_;
std::unique_ptr<ProteinBuildContext> build_context_;
std::vector<std::unique_ptr<ProteinConformation>> conformations_;
```

The direct storage point for bonds and rings is not `Protein`. In current code `Protein` delegates `BondAt`, `Bonds`, `RingAt`, `Rings`, and the saturated-ring accessors through `LegacyAmberTopology`. `Protein::BondCount()` and `Protein::RingCount()` return zero before topology exists; the bond and ring element/list accessors require the topology and abort if it is missing.

4.1 Atoms, residues, bonds, and rings

`Atom` is a flat identity record: `element`, display/serialization name `pdb_atom_name`, `residue_index`, `bond_indices`, and `parent_atom_index`. The methods `CovalentRadius`, `Electronegativity`, `IsHBondDonorElement`, and `IsHBondAcceptorElement` answer element-derived questions. There is no atom subclass hierarchy.

`Residue` stores amino-acid identity, sequence address, atom indices, protonation variant, terminal-state metadata, cached backbone slots `N`, `CA`, `C`, `O`, `H`, `HA`, `CB`, and up to four `ChiAtoms` records. It answers residue-level questions through methods such as `IsAromatic`, `IsTitratable`, `HasAmideH`, and `ChiAngleCount`. Backbone adjacency is not inferred from chain labels or sequence numbers; `Protein::BackbonePredecessor`, `BackboneSuccessor`, and `BackboneConnected` walk the peptide bond graph.

`Bond` stores atom endpoints, `BondOrder`, `BondCategory`, and `is_rotatable`. Its geometry methods take a position vector because midpoint, length, and direction depend on a conformation. The topology stays invariant; geometry is computed from the coordinate set supplied at the call site.

`Ring` is a virtual type hierarchy. A ring stores vertex atom indices, `RingTypeIndex`, parent residue, parent residue number, and fused partner. The type-specific physics is virtual: `Intensity`, `LiteratureIntensity`, `JohnsonBoveyLobeOffset`, `NitrogenCount`, `Aromaticity`, `RingAtomCount`, and `TypeName`. The current classes include `PheBenzeneRing`, `TyrPhenolRing`, `TrpBenzeneRing`, `TrpPyrroleRing`, `HisImidazoleRing`, `HidImidazoleRing`, `HieImidazoleRing`, `IndolePerimeterRing`, and the saturated `ProPyrrolidineRing`. The aromatic-type boundary is `kAromaticRingTypeCount = 8`; `ProPyrrolidine` is the first saturated type.

4.2 Topology construction order

Every loader must call `Protein::FinalizeConstruction` after adding atoms and residues and before adding any conformation. In source order, the method does the following:

```
ResolveResidueTerminalStates()
CacheResidueBackboneIndices()
ResolveProtonationStates(nullptr)
CovalentTopology::Resolve(atoms_, residues_, positions, bond_tolerance)
```

```
if disulfide authority exists:
  bonds->OverrideDisulfides(authority_pairs)
ResolveProtonationStates(bonds.get())
copy bond indices and hydrogen parents back onto Atom
run post-protonation NamingApplicator pass
ComposeAtomSemantic(atoms_, residues_, *bonds)
RingTopology::ConstructFromSubstrate(residues_, atom_semantic, *bonds)
bonds->TagAromaticBonds(rings->Aromatic())
construct LegacyAmberTopology(...)
CacheResidueBackboneIndices_Typed()
```

This order matters. The first backbone cache is a loader-boundary string pass; the final cache is substrate-driven and overwrites the earlier slots. Ring construction happens after protonation and semantic composition, and aromatic bond tagging is an overlay after rings exist. `DetectAromaticRings` is no longer part of the path.

5 Conformation scope

`ProteinConformation` is the geometry object. Its constructor takes a borrowed `Protein*`, a `std::vector<Vec3>` of positions, and an optional description. It moves the positions into `positions_`, then constructs `atoms_` as one `ConformationAtom` per position. The `ConformationAtom` constructor is private and `ProteinConformation` is its friend, so loose per-conformation atom records cannot be created outside the owning conformation.

The conformation stores:

- `positions_`, returned by `Positions`;
- `atoms_`, the parallel `ConformationAtom` vector whose position copy is read by `PositionAt`;
- `results_`, an unordered map keyed by `std::type_index` and owning `ConformationResult` pointers;
- ring geometry, ring pair records, bond lengths, bond directions, bond midpoints, bounding box, center of geometry, radius of gyration, and pre-built maps by ring type, bond category, and residue type;
- `geometry_choices`, the recorded geometric decisions made during `Compute`.

The geometry boundary is therefore explicit. Static identity is read through `conf.ProteinRef().AtomAt(i)` and topology through `conf.ProteinRef().LegacyAmber()`. Coordinate and computed data are read from `conf.PositionAt(i)` and `conf.AtomAt(i)`.

5.1 Conformation subclasses

`ProteinConformation` is not abstract. It is the complete working geometry object. The subclasses only add provenance:

`CrystalConformation` resolution, R factor, temperature, PDB ID.

`PredictionConformation` prediction method and confidence.

`MDFrameConformation` walker, time in ps, Boltzmann weight, RMSD, and radius of gyration.

`DerivedConformation` derivation description.

`Protein::AddDerived` does not store the parent argument; the current code uses it only to select the factory signature.

5.2 ConformationResult attachment

`ConformationResult` is the base class for computed conformation-scope results:

```
virtual std::string Name() const = 0;
virtual std::vector<std::type_index> Dependencies() const = 0;
virtual int WriteFeatures(const ProteinConformation&, const std::string&) const;
```

`ProteinConformation::AttachResult` checks three facts:

- the pointer is not null;
- no result of the same dynamic type is already attached;
- every declared dependency is already present in `results_`.

If the null check fails, `AttachResult` returns false. Duplicate or missing-dependency failures are rejected with a warning. If the checks pass, the result is stored under `typeid(*result)`. The `Result<T>` accessor returns the typed result and aborts if it is absent; `HasResult<T>` is the non-aborting presence check. The code therefore uses typed access, not string dispatch on result names.

Computation happens before attachment, in each subclass's static `Compute` factory. Attachment validates the singleton and dependency contract; it does not run the calculator.

5.3 ConformationAtom as the computed-data row

`ConformationAtom` is the per-atom computed-data row for one conformation. Its immutable field is the constructor-set `position_`. Everything else is typed, public calculator output. Important groups include:

- enrichment fields: `role`, `hybridisation`, `is_backbone`, `is_amide_H`, `is_hbond_donor`, and related booleans;
- force-field and MOPAC charges, PB radii, orbital populations, Wiberg neighbours;
- `spatial_neighbours`, `ring_neighbours`, and `bond_neighbours`;
- ring-current totals, McConnell totals, Coulomb fields, MOPAC Coulomb fields, APBS fields, H-bond fields, ring-based shielding kernels, graph fields, ORCA DFT tensors, tripeptide and Larsen tensors, AIMNet2 tensors, planar geometry, SASA, water fields, hydration geometry, EEQ charges, and prediction fields.

The per-source structured records are real types: `RingNeighbourhood`, `BondNeighbourhood`, `AtomNeighbour`, and `MopacBondNeighbour`. `RingNeighbourhood` stores the ring identity, ring-frame geometry, and per-ring calculator tensors. `BondNeighbourhood` stores bond identity, bond category, midpoint geometry, dipolar tensor, its spherical decomposition, and the McConnell scalar.

6 Tensor representation

The tensor output is preserved end to end. `Types.h` defines `Vec3` and `Mat3` as Eigen types and `SphericalTensor` as:

```
double T0;
std::array<double, 3> T1;
std::array<double, 5> T2;
```

`SphericalTensor::Decompose` maps a real 3×3 tensor into irreducible components: isotropic T_0, antisymmetric pseudovector T_1, and symmetric traceless T_2 in the code's real isometric basis. `PackFull9` emits the order

$$[T_0, T_{1x}, T_{1y}, T_{1z}, T_{2,-2}, T_{2,-1}, T_{2,0}, T_{2,+1}, T_{2,+2}].$$

`PackT2` exists for sources that are structurally pure T_2. `T2CosineWith` is a method on `SphericalTensor`; this is another example of an object answering a physics question about itself rather than shipping its arrays to a string-selected helper.

The wide `ConformationAtom` row contains both matrices and decomposed tensors when a calculator produces both. Examples include `total_G_tensor` with `total_G_spherical`, `coulomb_EFG_total` with `coulomb_EFG_total_spherical`, `apbs_efg` with `apbs_efg_spherical`, ORCA dia/para/total matrix+spherical pairs, and Larsen tensor+spherical pairs. Classical kernel totals such as `bs_shielding_contribution`, `mc_shielding_contribution`, and `coulomb_shielding_contribution` are stored as `SphericalTensor`; this model does not scalarize them at storage time.

7 OperationRunner

`OperationRunner` is the one place that runs the ordered conformation-level sequence, and it holds no state of its own. It takes a mutable `ProteinConformation` and a `RunOptions` value, computes the result objects in order, and attaches each to the conformation. A result is self-contained: a subclass declares its name and its type-index dependencies, holds the query surface and the typed output for its one computation, and is retrieved by its type, not by a string name. A calculator that depends on another result reads it from the conformation by type; there is no shared property store.

The current `RunOptions` surface is larger than the early design notes: charge source and net charge; skip flags for DSSP, MOPAC, APBS, and vacuum Coulomb; an AIMNet2 model pointer; GROMACS frame energy; solvent; bonded parameters; per-frame velocities and box matrix; ORCA NMR path; tripeptide DFT table; and Larsen H-bond grid.

The exact `OperationRunner::Run` order is:

```
GeometryResult
SpatialIndexResult
EnrichmentResult
PlanarGeometryResult        // soft attach; logs and continues on null
DsspResult                  // unless skip_dssp; soft external-tool policy
ChargeAssignmentResult      // if charge_source exists

if ChargeAssignmentResult attached:
  MopacResult               // unless skip_mopac; soft failure
  ApbsFieldResult           // unless skip_apbs; soft failure

BiotSavartResult
HaighMallionResult
McConnellResult
```

```
RingSusceptibilityResult
PiQuadrupoleResult
DispersionResult
CoulombResult               // if charges and !skip_coulomb
MopacCoulombResult          // if MopacResult attached
MopacMcConnellResult        // if MopacResult attached
HBondResult                 // if DsspResult attached
SasaResult
EeqResult
AIMNet2Result
AIMNet2ChargeResponseGradientResult
WaterFieldResult
HydrationShellResult
HydrationGeometryResult
GromacsFramePullResult
GromacsEnergyResult
BondedEnergyResult
TripeptideBackboneShieldingResult
TripeptideNeighborShieldingResult
LarsenHBondShieldingResult
OrcaShieldingResult         // optional; non-empty path failing is fatal
```

Some entries are hard requirements once requested. AIMNet2, explicit solvent calculators, GROMACS frame pull, energy, bonded energy, and the tripeptide and Larsen resources use `TimedAttach` or `Attach` failure to stop the run. DSSP, MOPAC, APBS, and planar geometry are allowed to be absent for specific input/resource failures and leave their result absent rather than faking data.

`RunMutantComparison` runs `Run` on WT and ALA conformations and, if both have `OrcaShieldingResult`, computes `MutationDeltaResult` against the mutant and attaches the delta to the WT conformation. The mutant is an input to the computation, not the owner of the delta result.

8 Trajectory runtime

`Trajectory::Run` drives the trajectory run. It is the only code path with mutable access to frame-zero conformation through the private `TrajectoryProtein::MutableCanonicalConformation_` friend hook. Public `TrajectoryProtein` exposes `ProteinRef` and `CanonicalConformation` as const, so `TrajectoryResult` subclasses cannot mutate the invariant protein or the per-instant conformation buffer through `TrajectoryProtein`.

8.1 Run phases

The eight phases in `Trajectory::Run` are:

1. Open `GromacsFrameHandler`.
2. Read frame 0 and call `TrajectoryProtein::Seed`.
3. Attach `TrajectoryResult`s from `RunConfiguration` factories, then caller extras.
4. Validate dependencies and session resources.
5. Build the base `RunOptions` template, then emit invariant atom/topology sidecars if an output directory is set.

6. Run frame 0 through `OperationRunner`, dispatch trajectory results, emit per-frame PDB/NPY, and record frame metadata.
7. For each later dispatched frame: skip according to stride, read, update `TrajectoryEnv`, create a tick conformation, run `OperationRunner`, dispatch trajectory results, emit, and record.
8. Finalize all trajectory results, mark complete, and release the handler.

Dependency validation is split deliberately. `TrajectoryProtein::AttachResult` checks only singleton-per-type. Phase 4 checks each `TrajectoryResult::Dependencies()` entry against either an attached trajectory result or the `RunConfiguration` set of required per-frame `ConformationResult` types. AIMNet2 is validated as a `Session` resource if the configuration requires it.

8.2 Frame environment

`TrajectoryEnv` is a single-slot frame environment. It stores solvent, current energy pointer, frame index, frame time, velocities, and box matrix. `Trajectory::Run` overwrites it before each dispatched frame. Results that need history copy what they need into their own state during `Compute`; `TrajectoryEnv` is not a history buffer.

9 Buffer management

The system's buffer discipline is constructor allocation with scoped ownership: a buffer is allocated by the object whose lifetime defines its validity, when that object is built. The sizes are known at that point — the atom count, the per-atom stride — so nothing is grown or attached afterward. A conformation sizes its atom rows from the position vector it receives; a `TrajectoryProtein` sizes its per-atom records once the seed frame fixes the atom count; on creation, a `DenseBuffer` holds the atom count and the stride per atom. The lifetime defines the validity; there is no register/release step and no detach.

9.1 Conformation buffers

`ProteinConformation` owns `positions_` and the parallel `atoms_` vector. Both are sized in the constructor from the input position vector. A `ConformationAtom` contains its own position copy as `position_`; `PositionAt(i)` returns that value, while `Positions()` returns the source vector. Calculators write typed fields onto the `ConformationAtom` row.

`ConformationResult` objects are owned by the conformation under `std::unique_ptr`. There is no separate property-store framework. The older design idea of a typed store interface does not exist in current code; result `Compute` methods write the named fields directly.

9.2 TrajectoryAtom buffers

`TrajectoryProtein::Seed` finalizes the wrapped protein, seats frame 0, and then calls `InitTrajectoryAtoms`. That method clears and reserves the vector to `protein_->AtomCount()` and constructs one `TrajectoryAtom` per atom. `TrajectoryAtom` has a private default constructor and `TrajectoryProtein` is its friend, so the trajectory atom buffer is only created by the owning `TrajectoryProtein`.

The current `TrajectoryAtom` stores named Welford state substructs: `bs_welford`, `hm_welford`, `mc_welford`, `eeq_welford`, `sasa_welford`, `hbond_count_welford`, `water_field_welford`,

hydration_geometry_welford, hydration_shell_welford, aimnet2_charge_response_gradient_welford, and mopac_charge_welford. Each substruct has one writer trajectory result. The atom also owns RecordBag<AtomEvent> events.

9.3 DenseBuffer

DenseBuffer<T> is the finalize-only time-series and lag buffer. It is constructed with atom_count and stride_per_atom. Its storage is a single std::vector<T> of length $atom\_count \times stride\_per\_atom$, laid out atom-major:

```
storage[atom_idx * stride_per_atom + offset]
```

The result that needs the buffer usually owns it while frames are being processed. At Finalize, it transfers ownership to TrajectoryProtein::AdoptDenseBuffer<T>, keyed by the owning result's std::type_index. GetDenseBuffer<T> returns null if the owner key is absent or if the stored element type differs. This keeps mixed buffer payloads behind DenseBufferBase without losing typed access at use sites.

H5 writers flatten DenseBuffer content by explicit component access rather than by reinterpreting raw memory. This is important for Eigen-backed types and for SphericalTensor; layout in memory is not the serialization contract.

10 RunConfiguration

RunConfiguration is a typed named run shape. A shape holds: name_, per_frame_opts_, deferred trajectory-result factories, required_conf_result_types_, requires_aimnet2_, and stride_. A deferred factory is a callable that takes a const TrajectoryProtein& and returns a std::unique_ptr<TrajectoryResult>. Factories are deferred because trajectory results size their buffers after TrajectoryProtein::Seed, when atom, bond, and ring counts are known.

The code currently exposes two named shapes:

PerFrameExtractionSet Production canonical trajectory shape. It skips MOPAC and vacuum Coulomb, keeps APBS and DSSP on, requires AIMNet2, requires the full per-frame conformation stack listed in RunConfiguration.cpp, and attaches the Welford, time-series, solvent, dynamics, geometry, energy, dihedral, ring, and selection trajectory results.

FullFatFrameExtraction Starts from PerFrameExtractionSet, sets skip_mopac=false and skip_coulomb=false, and appends the MOPAC-family trajectory results: MOPAC charge Welford, MOPAC bond-order Welford, MOPAC Coulomb shielding time series, MOPAC McConnell shielding time series, and MOPAC-vs-FF14SB reconciliation.

The current code does not set a buried stride in PerFrameExtractionSet; stride_ defaults to 1 and the caller's SetStride call is the single cadence knob.

11 Session and logging

Session is the process-resource object. It is not yet a complete instance replacement for every static subsystem; instead, LoadFromToml orchestrates startup in one named step:

```
RuntimeEnvironment::Load()
OperationLog::LoadChannelConfig()
```

```
OperationLog::LogSessionStart()
CategoryInfoProjection::Configure(...)
```

The same `Session` optionally owns `AIMNet2Model`, `TripeptideDftTable`, and `LarsenHBondGrid`. The trajectory run uses these through the const `Session` reference and copies the relevant pointers into the per-frame `RunOptions`.

`OperationLog` emits structured JSON with `ts`, `level`, `op`, and `detail`. It can write UDP, file, and stderr. Warning and error messages always emit; channelled info and debug messages obey the channel mask, while sink/session setup uses the unfiltered overload. The implementation is explicitly `noexcept` at the logging API boundary and contains failures internally, because logging is used in destructors and error paths. The code does not implement automatic per-property-store logging; there is no property-store operation in the current architecture.

12 Feature and H5 emission

`ConformationResult::WriteAllFeatures` creates the output directory, writes identity arrays, writes sparse ring-contribution rows, writes the ring geometry table, and then iterates all attached conformation results and calls their `WriteFeatures`. The attached-result iteration is unordered-map order; dependency order matters at compute/attach time, not at feature writing.

`Trajectory::Run` writes `CategoryInfoProjection` and `TopologySidecar` once before per-frame work when `output_dir` is set. `Trajectory::WriteH5` writes the `/trajectory` group, source path attributes, frame times, original frame indices, and selection records grouped by kind. `TrajectoryProtein::WriteH5` writes file-root protein attributes, the `/atoms` passthrough group, and then delegates to each attached `TrajectoryResult::WriteH5Group` in attach order.

The architectural rule is distributed ownership of output: the result that knows the semantics writes the arrays or H5 group for those semantics.

13 Load-bearing invariants

13.1 Protein is identity and topology only

The code enforces this boundary in three places. `Atom` has no position. `Bond` geometry methods take an external position vector. `Protein` has no `ConformationAtom` member and no result map; those live inside each `ProteinConformation`. Per-atom geometry lives on the conformation and its `ConformationAtom` rows; per-frame trajectory history lives on `TrajectoryAtom` or result-owned `DenseBuffer`s.

13.2 Tensor output is preserved

`SphericalTensor` is the common decomposition type. Calculator outputs in `ConformationAtom` retain matrix+spherical pairs when a matrix is produced, and kernel totals preserve the decomposed `T0/T1/T2` structure. Time-series trajectory results use `DenseBuffer<SphericalTensor>` for full nine-component outputs, and T2-only emission only where the source tensor is structurally traceless.

13.3 Objects answer typed questions about themselves

The runtime calculator surface does not identify chemistry by string matching. `Ring` answers `Intensity`, `Aromaticity`, and `NitrogenCount`. `Bond` answers `IsPeptideBond`, `IsBackbone`, and `IsAromatic`. `Residue`

answers `IsAromatic` and `ChiAngleCount`. `Protein` answers backbone connectivity by walking typed peptide bonds. `SphericalTensor` answers packing, reconstruction, magnitude, inner product, and signed T2 cosine.

String work remains at construction and naming boundaries where it belongs: loader atom names, early backbone cache, the post-protonation naming pass, and output naming projection. After construction, calculators consume typed objects and typed fields.

14 Design-doc drift noted from the source

- Some design text says `Protein` owns bonds and rings directly. Current code stores them under `LegacyAmberTopology` through `CovalentTopology` and `RingTopology`; `Protein` exposes delegating accessors.
- Some trajectory-scope design text describes `PerFrameExtractionSet` as stride 2. Current `RunConfiguration.cpp` leaves stride at default 1 and states that caller `SetStride` is the single cadence knob.
- Several design sections describe every MD frame as an `MDFrameConformation`. Current trajectory code seats only frame 0 through `AddMDFrame`. Later frames use free-standing base `ProteinConformation` objects from `TickConformation`.
- Older Session descriptions mention only AIMNet2. Current `Session` also owns optional `TripeptideDftTable` and `LarsenHBondGrid` resources and passes them into `OperationRunner` through `RunOptions`.
- Older property-store and UDP-log sketches describe an automatic typed store interface. Current code has direct field writes by result `Compute` methods and explicit `OperationLog` calls; there is no `StoreAtomProperty` or automatic per-field store logger.
- The conformation-level pipeline in the code has grown beyond the shorter classical-stack descriptions: `PlanarGeometryResult`, `EeqResult`, AIMNet2 charge-response gradient, explicit water and hydration results, GROMACS frame pull, bonded energy, tripeptide shielding, and Larsen H-bond shielding are all part of the current `OperationRunner::Run` surface when their options/resources are present.